

MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Balan Gabriel FAF-241

Report

Laboratory Work No. 4

Operating Systems

Topic: The xv6 Shell from the Outside

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

1 Purpose of the Work 3

2 Part 1 – Fix Sleep for the Current xv6 3

3 Part 2 – Bash versus the xv6 Shell 5

3.1 Observations 5

4 Part 3 – Who Expands *? A Program that Prints its argv 5

4.1 Observations 8

5 Part 4 – Read the Shell 11

5.1 Observations 11

6 Conclusion 13

1. Purpose of the Work

The purpose of this laboratory work is to compare the xv6 shell with Bash, observe which shell features xv6 implements and which features Bash adds on top, examine how command-line arguments are passed to programs, and read the xv6 shell source code to connect its visible behavior with its implementation.

2. Part 1 – Fix Sleep for the Current xv6

The whole thing I had already done in Lab 2. I noticed the inconsistency in naming when trying to run `sleep`, checked the system calls, and saw that the proper system call in the current tree is `pause`. Nothing had to be changed at this point; the command ran perfectly.

I first checked which system call name is declared in `user/user.h`:

Checking the current system call

```
grep -n "pause\|sleep" user/user.h
```

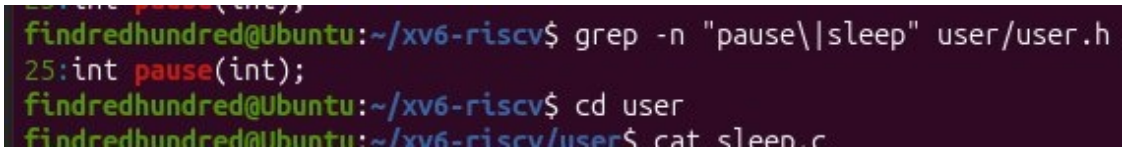


Figure 1.1 – The `grep` result in `user/user.h`, showing that the current xv6 tree uses `pause`.

The `sleep.c` program already uses `pause(atoi(argv[1]))` and checks that exactly one argument is provided.

user/sleep.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // system call into the kernel
  exit(0);
}
```

After building and booting xv6, I tested the program both with and without the required argument.

```
findredhundred@Ubuntu:~/xv6-riscv/user$ cat sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // system call into the kernel
  exit(0);
}
```

Figure 1.2 – The existing `sleep.c` program using the `pause` system call.

```
findredhundred@Ubuntu:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global vi
rtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,driv
e=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sleep 20
$ sleep
usage: sleep <ticks>
```

Figure 1.3 – The `sleep` program running in xv6 with an argument and showing its usage message without one.

3. Part 2 – Bash versus the xv6 Shell

I ran the same commands in Bash and in the xv6 shell and compared their output. The results make it clear which features are basic shell functionality and which conveniences are provided by Bash but are not implemented in xv6.

Command	Bash	xv6
`echo hello > f ; cat < f`	`hello`	`hello`
`ls | grep READ`	`README`	`README 2 2 2441`
`(echo a; echo b) | wc`	`2 2 4`	`2 2 4`
`echo one ; echo two`	`one` then `two`	`one` then `two`
`echo bg &`	job number, `bg`, then job completion	`bg`
`ls *`	file names such as `LICENSE`, `Makefile`, etc.	`ls: cannot open *`
`echo $HOME`	`/home/findredhundred`	`$HOME`
`echo "a b"`	`a b`	`"a b"`
`mkdir d ; cd d ; ls ..`	parent directory listing	`ls` no longer runs normally from inside `d`
Arrow up, Tab, Ctrl+R	works	does not work

3.1. Observations

1. Both shells support executing commands, input/output redirection, pipes, sequences, grouped commands, and background execution. Bash additionally provides wildcard expansion, environment-variable expansion, quote handling, PATH-based program lookup, command history, Tab completion, and reverse history search. The exact xv6 build may provide a subset of extra conveniences.

2. After `cd d`, xv6 tries to find `ls` in the current directory. Standard xv6 user programs reside in the filesystem root, and its shell does not search PATH automatically. Running `../ls ..` (or `/ls ..`) from `d` invokes the program successfully while remaining inside `d`.

3. I would miss command history and Tab completion most, because they avoid retyping commands and reduce errors in daily work. Basic variable expansion, such as recognizing `$HOME`, would be relatively easy to add compared with a full interactive history and completion editor.

4. Part 3 – Who Expands *? A Program that Prints its argv

To see exactly what arguments a program receives, I used a small program that prints `argc` and every element of `argv`. I used one version for xv6 and the corresponding Linux version compiled with GCC.

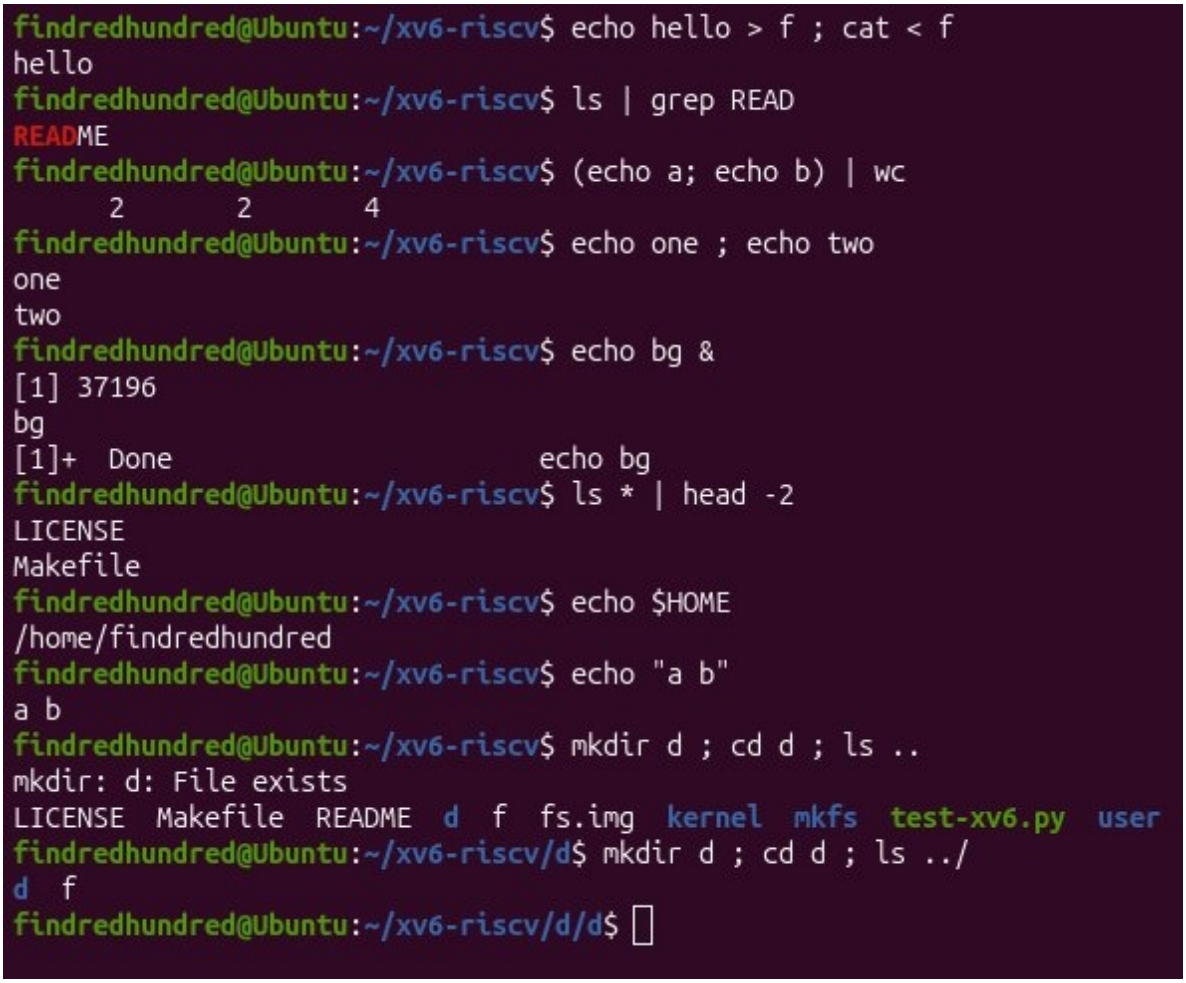


Figure 2.1 – The comparison commands run in Bash.

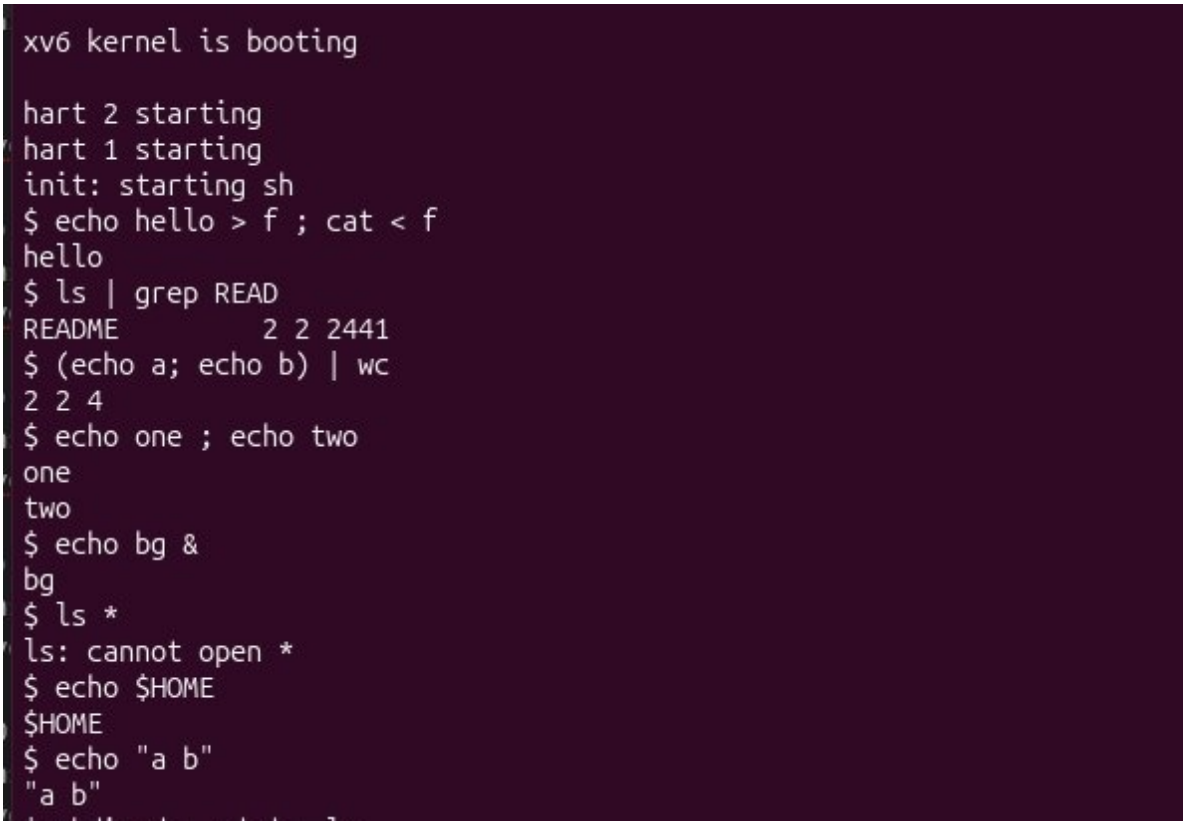


Figure 2.2 – The same shell-feature tests run in xv6.



Figure 2.3 – After changing into `d`, xv6 cannot find `ls` normally; using `../ls ..` runs it from the parent directory.

Core argument-printing logic

```
int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  return 0;
}
```

```
findredhundred@Ubuntu:~/os-lab3/tmp$ ls
args  args_linux.c  ayaya.txt  meh.sh  test1.c  test8.c
findredhundred@Ubuntu:~/os-lab3/tmp$ cat args_linux.c
#include <stdio.h>
int
main(int argc, char *argv[])
{
printf("argc = %d\n", argc);
for(int i = 0; i < argc; i++)
printf("argv[%d] = [%s]\n", i, argv[i]);
return 0;
}
findredhundred@Ubuntu:~/os-lab3/tmp$ in MAKEFILE inserted $U/_args\
```

Figure 3.1 – The Linux argument-printing program and the corresponding `args` program added to the build.

In xv6, the command-line text is passed much more literally:

The same tests in Linux show Bash performing expansion before starting the program:

4.1. Observations

1. When executing `args *.c "x y"` in xv6, the program received 4 arguments, including the program name. The output was `argv[0] = [args]`, `argv[1] = [*.c]`, `argv[2] = ["x]`, and `argv[3] = [y"]`. This shows that xv6 did not expand the wildcard or recognize the quotation marks as grouping characters. In Bash, the same experiment using `./args` produced 5 arguments: `argv[0] = [./args]`, `argv[1] = [args_linux.c]`, `argv[2] = [test1.c]`, `argv[3] = [test8.c]`, and `argv[4] = [x y]`. Bash expanded `*.c` into the three matching C files and passed the quoted text as a single argument without quotation marks.

2. The shell is responsible for expanding wildcards into filenames and processing quotation marks before executing a program. These transformations are not performed by the kernel or by the argument-printing program itself. The experiment demonstrated this because the same program logic received different arguments depending on which shell executed it. Bash performed wildcard expansion and quote removal, while the xv6 shell passed the wildcard and quotation marks as literal characters.

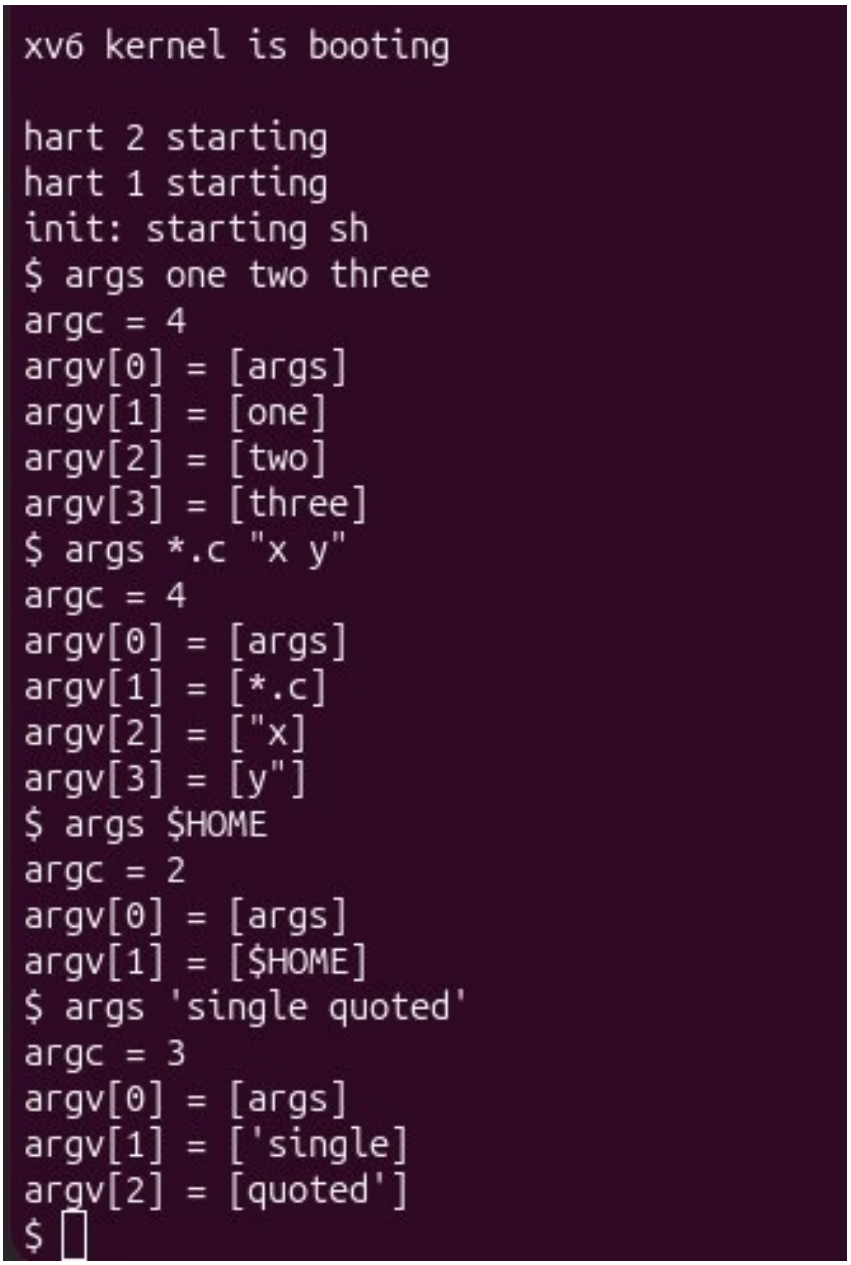


Figure 3.2 – Output of `args` in xv6. Wildcards and quotation marks are not processed like they are in Bash.

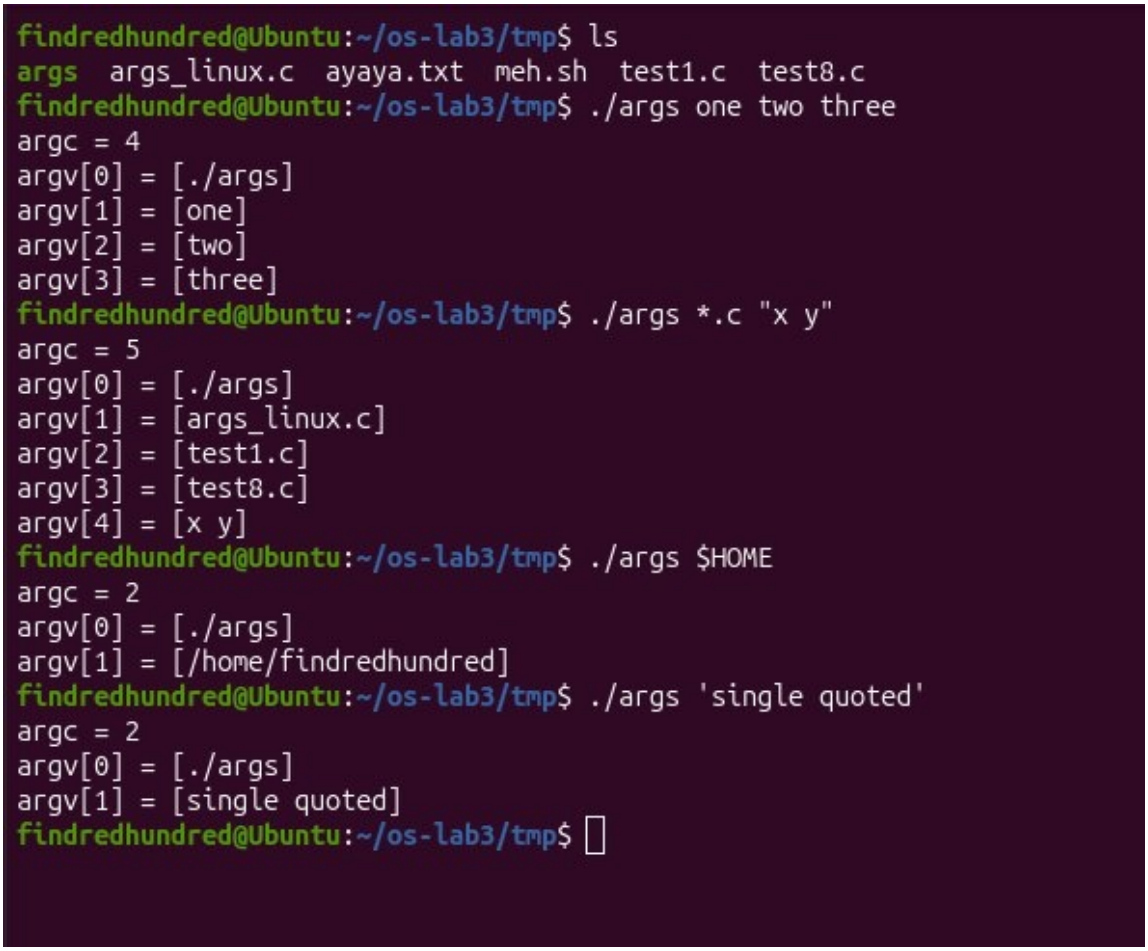


Figure 3.3 – Output of the Linux version of `args` in Bash.

3. When executing `args one two three` with many extra spaces between the words, the extra spaces were removed during command-line parsing. Both Bash and xv6 treated consecutive unquoted spaces as separators rather than as part of the arguments. Consequently, both programs received 4 arguments, including the program name: `args`, `one`, `two`, and `three`. This demonstrates that both shells perform basic whitespace-based argument splitting before starting a program.

5. Part 4 – Read the Shell

Finally, I inspected `user/sh.c` directly. The file has 499 lines in my tree, and the command-type definitions appear near the beginning of the file.

Inspecting user/sh.c

```
wc -l user/sh.c
grep -n "#define" user/sh.c
grep -n "case " user/sh.c
grep -n "cd" user/sh.c | head
```



Figure 4.1 – The five command kinds defined near the beginning of `user/sh.c`.

5.1. Observations

1. The xv6 shell recognizes five command types: `EXEC`, `REDIR`, `PIPE`, `LIST`, and `BACK`. The `EXEC` type represents the execution of a normal command and does not require a special operator. The `REDIR` type handles input and output redirection using `<`, `>`, and `»`. The `PIPE` type is produced by the `|` character and connects the output of one command to the input of another. The `LIST` type is produced by `;` and allows multiple commands to execute sequentially. The `BACK` type is produced by `&` and allows a command to execute in the background. These five types are defined near the beginning of `user/sh.c` and are handled by the shell's execution and parsing functions.

2. The `cd` command is handled directly inside `user/sh.c`, in the main shell loop, rather than being executed as a separate program. In the source file, the relevant code appears around line 170. This is necessary because changing the working directory must affect the shell process itself. If `cd` were executed as a separate program, the directory change would affect only that child process, and the shell would remain in its original directory after the child terminated. Therefore, `cd` is implemented as a built-in shell command.

```
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
findredhundred@Ubuntu:~/xv6-riscv$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
```

Figure 4.2 – Locations of the `case` branches used to parse and execute the different command kinds.

```
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
findredhundred@Ubuntu:~/xv6-riscv$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
```

Figure 4.3 – The source location showing that `cd` is handled by the shell itself.

3. The source code of `user/sh.c` does not contain functionality for wildcard expansion using `*` or environment-variable expansion using `$`. This explains the behavior observed in Part 2, where `ls *` failed because xv6 treated the asterisk as a literal filename, and `echo $HOME` printed `$HOME` instead of the user's home directory. Unlike Bash, the xv6 shell does not perform these substitutions before executing commands. Its command parser supports basic execution, redirection, pipes, command lists, and background execution, but it does not implement the more advanced expansion features provided by Bash.

6. Conclusion

This laboratory work made the difference between a small teaching shell and a modern interactive shell much more visible. xv6 already has the core mechanisms needed to execute programs, redirect input and output, connect commands with pipes, run command lists, and start background commands. Bash adds many conveniences on top of those basics, especially wildcard expansion, variable expansion, quote processing, PATH lookup, history, and completion.

The argument-printing experiment also showed clearly that many transformations we usually associate with a command are actually done by the shell before the program starts. Reading `user/sh.c` connected those visible behaviors to the implementation: the xv6 shell has a small number of command types and directly handles special cases such as `cd`. Overall, the lab helped me understand not only what a shell does from the outside, but also why its design has to work that way internally.